

Experiments with PROMISE using customized precisions

Xinye Chen, Thibault Hilaire, Fabienne Jézéquel

This document presents:

- in Section 1, experimental results obtained with the precision autotuning tool PROMISE [3] [4] extended this support to FP16 precision, while subsequent work [2] <https://gitlab.lip6.fr/pequan/promise> using customized precisions
- in Section 2, the details of the tested algorithms.

The benchmark code is publicly available at <https://github.com/PEQUAN/hpc-mix-mlbench>.

1 Empirical results

Our simulation tasks as well as their categorization are presented in Table 1. PROMISE is run on algorithms from the Rodinia heterogeneous computing benchmark suite¹ and on well-established linear solving methods.

PROMISE is evaluated across four precision combinations (I–IV) as depicted in Table 2 to examine the number of variables of each type in the transformed codes. In our experiments the number of required digits in the results ranges from 1 to 10. The following figures also present the computation time of PROMISE for each required accuracy, including the time to compute the reference result and the time to apply the delta debugging algorithm several times, compiling and executing the code with the tested distribution each time.

¹<https://rodinia.cs.virginia.edu/>

Table 1: Algorithms used for Benchmarking.

Benchmark	Domain	Characteristics
Back Propagation	Machine Learning	Neural network training via back-propagation. Compute-intensive, unstructured grid operations.
HotSpot	Physics Simulation	2D thermal simulation for chip design. Compute-intensive, structured grid, stencil-based computations.
ParticleFilter	Medical Imaging	Probabilistic object tracking in videos. Irregular memory access, compute-heavy, suitable for parallelization.
SRAD	Image Processing	Noise reduction in images while preserving edges. Structured grid, compute-intensive, iterative.
LU Factorization	Linear Algebra	Matrix decomposition via Gaussian elimination for solving linear systems. Our benchmarks include dense LU and sparse LU.

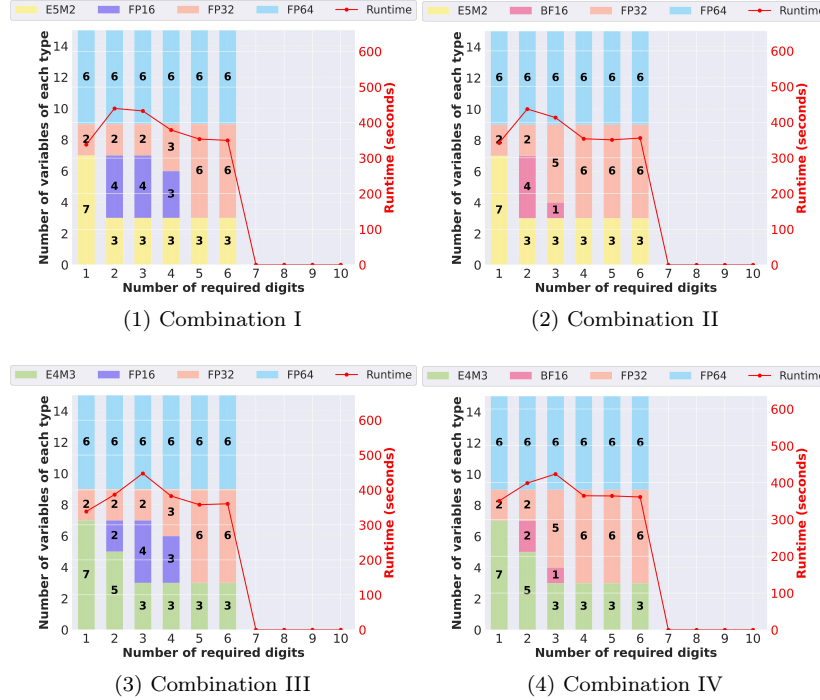


Figure 1: Precision configurations for Backprop with varying requested accuracies.

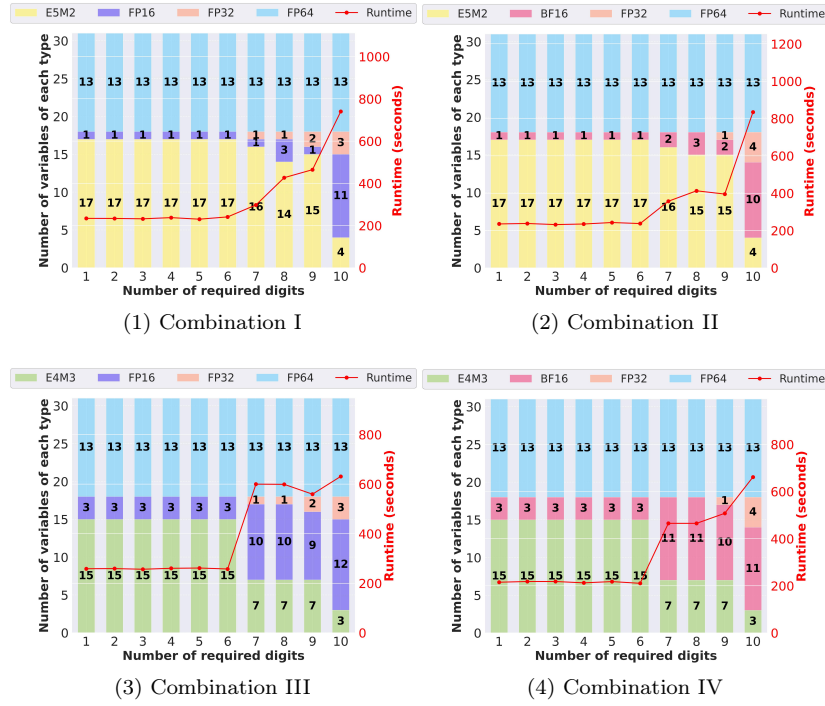


Figure 2: Precision configurations for Hotspot with varying requested accuracies.

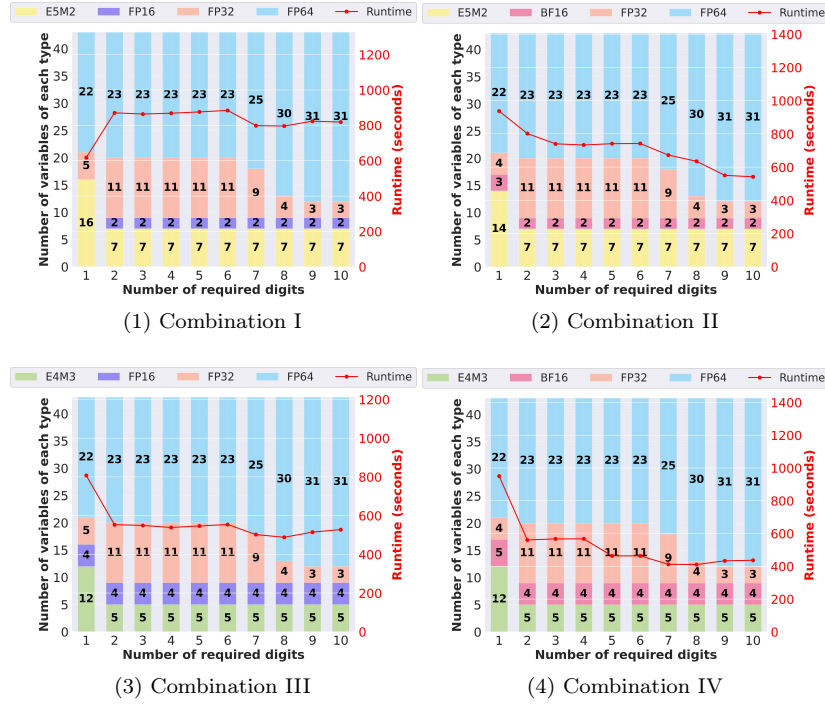


Figure 3: Precision configurations for Particle Filter with varying requested accuracies.

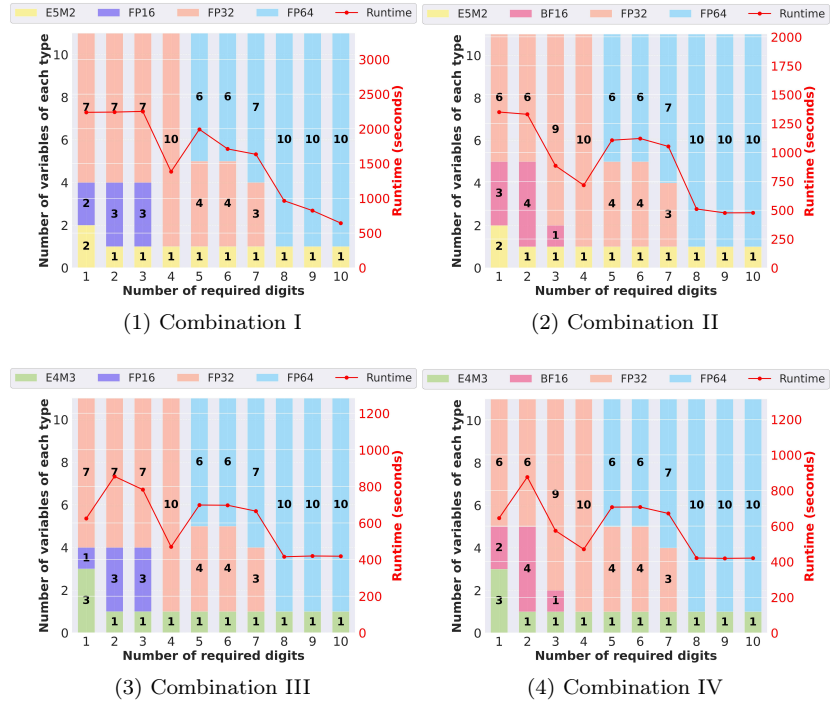


Figure 4: Precision configurations for SRAD with varying requested accuracies.

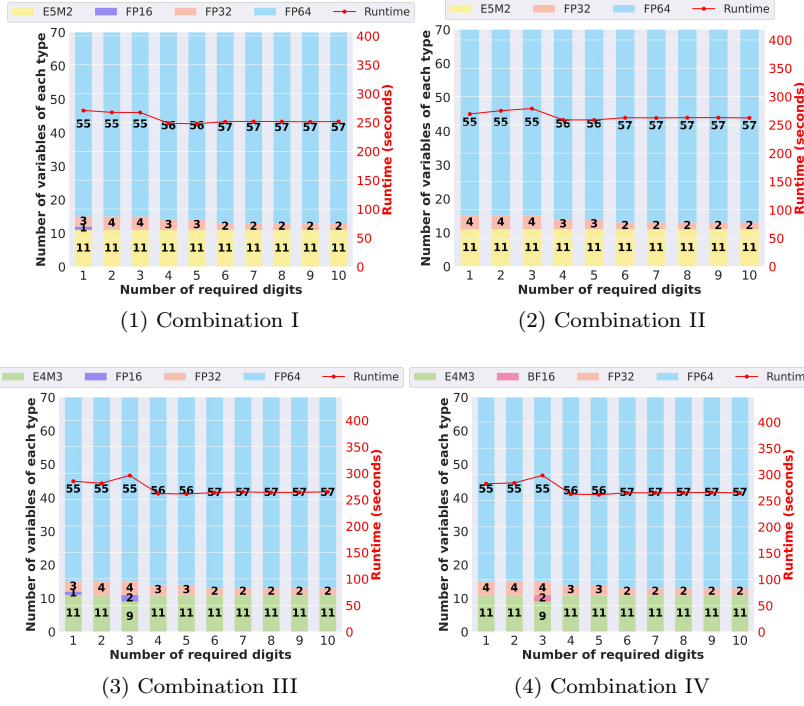


Figure 5: Precision configurations for dense LU factorization with varying requested accuracies.

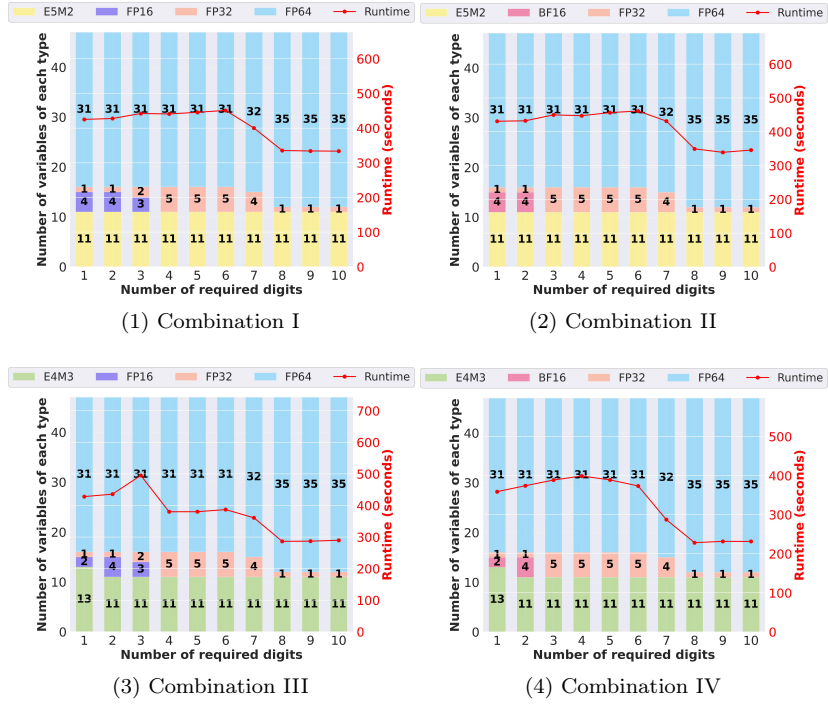


Figure 6: Precision configurations for sparse LU factorization with varying requested accuracies.

Combination	Format 1	Format 2	Format 3	Format 4
I	E5M2	FP16	FP32	FP64
II	E5M2	BF16	FP32	FP64
III	E4M3	FP16	FP32	FP64
IV	E4M3	BF16	FP32	FP64

Table 2: Floating-point format combinations used in the study

2 Algorithm pseudocode

We present here the algorithm pseudocode of Backprop, HotSpot, SRAD, Particle Filter and LU factorization.

Algorithm 1 Rodinia Backprop Kernel [1, 5]

Require: Input vector $\mathbf{x} \in \mathbb{R}^{n_{\text{in}}+1}$ $\triangleright$ input[0]=1.0 (bias unit), target vector $\mathbf{t} \in \mathbb{R}^{n_{\text{out}}}$, input-to-hidden weights $\mathbf{W}^{(1)} \in \mathbb{R}^{n_{\text{hid}} \times (n_{\text{in}}+1)}$, hidden-to-output weights $\mathbf{W}^{(2)} \in \mathbb{R}^{n_{\text{out}} \times (n_{\text{hid}}+1)}$, learning rate $\eta > 0$, momentum $m = 0.9$, previous deltas $\Delta \mathbf{W}_{\text{old}}^{(1)}$, $\Delta \mathbf{W}_{\text{old}}^{(2)}$

Ensure: Network output $\mathbf{a}^{(2)} \in \mathbb{R}^{n_{\text{out}}}$ (before update), output error $\delta^{(2)}$, hidden error $\delta^{(1)}$

```

1: for  $j = 1$  to  $n_{\text{hid}}$  (over neurons) do  $\triangleright$  Forward: Input  $\rightarrow$  Hidden
2:    $z_j^{(1)} \leftarrow \mathbf{W}_{j,:}^{(1)} \cdot \mathbf{x}$   $\triangleright$  pre-activation accumulator (sigmoid input path)
3:    $a_j^{(1)} \leftarrow \sigma(z_j^{(1)})$ 
4: end for
5: for  $k = 1$  to  $n_{\text{out}}$  (over neurons) do  $\triangleright$  Forward: Hidden  $\rightarrow$  Output
6:    $z_k^{(2)} \leftarrow \mathbf{W}_{k,:}^{(2)} \cdot \mathbf{a}^{(1)}$   $\triangleright$  pre-activation accumulator (sigmoid input path)
7:    $a_k^{(2)} \leftarrow \sigma(z_k^{(2)})$ 
8: end for
9: for  $k = 1$  to  $n_{\text{out}}$  do  $\triangleright$  Backward: Output Layer
10:   $\delta_k^{(2)} \leftarrow (t_k - a_k^{(2)}) \cdot \sigma'(z_k^{(2)})$   $\triangleright$  error-delta computation
11: end for
12: for  $j = 1$  to  $n_{\text{hid}}$  do  $\triangleright$  Backward: Hidden Layer
13:   $e_j \leftarrow \sum_k \delta_k^{(2)} \cdot W_{kj}^{(2)}$   $\triangleright$  reduction accumulator in hidden_error
14:   $\delta_j^{(1)} \leftarrow e_j \cdot \sigma'(z_j^{(1)})$   $\triangleright$  error-delta computation
15: end for
16: for  $j = 1$  to  $n_{\text{hid}}$  do  $\triangleright$  Update with Momentum + Bias Unit
17:   for  $i = 0$  to  $n_{\text{in}}$  do
18:     $\text{new\_dw} \leftarrow \eta \cdot \delta_j^{(1)} \cdot x_i + m \cdot \Delta W_{\text{old},j,i}^{(1)}$   $\triangleright$  new_dw in adjust_weights
19:     $W_{j,i}^{(1)} \leftarrow W_{j,i}^{(1)} + \text{new\_dw}$ 
20:     $\Delta W_{\text{old},j,i}^{(1)} \leftarrow \text{new\_dw}$ 
21:   end for
22: end for
23: for  $k = 1$  to  $n_{\text{out}}$  do
24:   for  $j = 0$  to  $n_{\text{hid}}$  do
25:     $\text{new\_dw} \leftarrow \eta \cdot \delta_k^{(2)} \cdot a_j^{(1)} + m \cdot \Delta W_{\text{old},k,j}^{(2)}$   $\triangleright$  new_dw in adjust_weights
26:     $W_{k,j}^{(2)} \leftarrow W_{k,j}^{(2)} + \text{new\_dw}$ 
27:     $\Delta W_{\text{old},k,j}^{(2)} \leftarrow \text{new\_dw}$ 
28:   end for
29: end for
30: return

```

Algorithm 2 Rodinia HotSpot Kernel (One Iteration) [1]

Require: Temperature grid $\mathbf{T}_{\text{in}} \in \mathbb{R}^{\text{rows} \times \text{cols}}$, Power grid $\mathbf{P} \in \mathbb{R}^{\text{rows} \times \text{cols}}$, Precomputed scalars: Cap (cell capacitance), R_x, R_y, R_z (thermal resistances), $step$ (time step), $C_1 = step/C$, $R_{x1} = 1/R_x$, $R_{y1} = 1/R_y$, $R_{z1} = 1/R_z$

Ensure: Updated temperature grid $\mathbf{T}_{\text{out}}$ for this iteration

- 1: **for** $i = 0$ **to** $rows - 1$ (**parallel over rows**) **do**
- 2: **for** $j = 0$ **to** $cols - 1$ (**parallel over columns**) **do** $\triangleright$ Structured 5-point stencil with boundary handling
- 3: $T \leftarrow T_{\text{in}}[i, j]$
- 4: **if** internal point **then**
- 5: $T_N \leftarrow T_{\text{in}}[i - 1, j]$, $T_S \leftarrow T_{\text{in}}[i + 1, j]$
- 6: $T_W \leftarrow T_{\text{in}}[i, j - 1]$, $T_E \leftarrow T_{\text{in}}[i, j + 1]$
- 7: $\Delta \leftarrow (T_N - T) \cdot R_{y1} + (T_S - T) \cdot R_{y1} + (T_W - T) \cdot R_{x1} + (T_E - T) \cdot R_{x1} + P[i, j] + (T_{\text{amb}} - T) \cdot R_{z1}$ $\triangleright$ Laplacians = neighbor-difference terms scaled by reciprocal resistances (R_{x1} , R_{y1} , R_{z1})
- 8: **else**
- 9: Use reduced neighbors + ambient temperature term (boundary/corner cases)
- 10: **end if**
- 11: $\delta \leftarrow C_1 \cdot \Delta$ $\triangleright$ δ is the local temperature increment; C_1 , $step$, R_{x1} , R_{y1} , R_{z1} are the “fast” scalars
- 12: $T_{\text{out}}[i, j] \leftarrow T + \delta$
- 13: **end for**
- 14: **end for**

Algorithm 3 Rodinia Particle Filter Kernel [1]

Require: Video sequence (synthetic moving disk object + noise), $N_{\text{particles}}$ particles with state (x, y) , weights w_i , noise parameters (fixed drift + Gaussian)

Ensure: Final particle positions **arrayX**, **arrayY** $\in \mathbb{R}^{N_{\text{particles}}}$ (after all frames + resampling)

```
1: for  $k = 1$  to  $N_{\text{fr}}$  (over video frames) do
2:   for  $i = 1$  to  $N_{\text{particles}}$  (parallel over particles) do  $\triangleright$  Prediction: stochastic
      proposal with constant drift + Gaussian noise  $\mathcal{N}(0, 1)$ . Gaussian sampling is
      implemented internally via Box-Muller transform using log, square root, and cosine
3:      $x_i \leftarrow x_i + 1 + 5 \cdot \mathcal{N}(0, 1)$ 
4:      $y_i \leftarrow y_i - 2 + 2 \cdot \mathcal{N}(0, 1)$ 
5:   end for
6:   for  $i = 1$  to  $N_{\text{particles}}$  do  $\triangleright$  Likelihood (compute-heavy)
7:     Compute indices into image via disk neighbors (irregular access)
8:      $w_i \leftarrow w_i \cdot \exp(\text{calcLikelihoodSum}(I, \text{indices}, \text{countOnes}))$ 
9:   end for
10:  Normalize:  $w_i \leftarrow w_i / \sum w$ 
11:  Build CDF (Cumulative Distribution Function of normalized weights):
       $\text{CDF}[0] \leftarrow w_0, \quad \text{CDF}[i] \leftarrow \text{CDF}[i-1] + w_i$ 
12:  for  $i = 1$  to  $N_{\text{particles}}$  do  $\triangleright$  Resampling (irregular access)
13:    Draw systematic  $u_i = u_1 + i/N_{\text{particles}}$ 
14:    Find index  $j$  s.t.  $\text{CDF}[j] \geq u_i$ 
15:    Duplicate particle  $j$ :  $x'_i \leftarrow x_j, y'_i \leftarrow y_j$ 
16:  end for
17:  Update arrayX, arrayY to resampled particles; reset  $w_i = 1/N_{\text{particles}}$ 
18:  Compute weighted mean  $x_e, y_e$  (for monitoring only)
19: end for
```

Algorithm 4 Rodinia SRAD Kernel [1]

Require: Image $I \in \mathbb{R}^{rows \times cols}$ (randomized input), speckle region $(r1, r2, c1, c2)$, λ (diffusion coefficient), n_{iter} (number of iterations)

Ensure: Denoised image $J \in \mathbb{R}^{rows \times cols}$ (updated in-place)

- 1: $J[k] \leftarrow \exp(I[k])$ for all pixels ▷ log-domain conversion
- 2: **for** $iter = 1$ **to** n_{iter} **do**
- 3: Compute ROI statistics on J (only in speckle rectangle $(r1..r2, c1..c2)$):
 $sum \leftarrow \sum J$, $sum2 \leftarrow \sum J^2$
 $meanROI \leftarrow sum/size_R$, $varROI \leftarrow sum2/size_R - meanROI^2$
 $q_0^2 \leftarrow varROI/(meanROI \cdot meanROI)$
- 4: **for** each pixel (i, j) (**parallel**) **do** ▷ Stage 1: directional derivatives + diffusion coeff.
- 5: $J_c \leftarrow J[k]$, $k = i \cdot cols + j$ ▷ center pixel value in working image J
- 6: $dN[k] \leftarrow J[iN[i] \cdot cols + j] - J_c$ ▷ (similarly for dS, dW, dE)
- 7: $G^2 \leftarrow (dN^2 + dS^2 + dW^2 + dE^2)/J_c^2$
- 8: $L \leftarrow (dN + dS + dW + dE)/J_c$
- 9: $q^2 \leftarrow (0.5 \cdot G^2 - (1/16) \cdot L^2)/(1 + 0.25 \cdot L)^2$
- 10: $c[k] \leftarrow 1/(1 + (q^2 - q_0^2)/(q_0^2 \cdot (1 + q_0^2)))$
- 11: Saturate: $c[k] \leftarrow \max(0, \min(1, c[k]))$
- 12: **end for**
- 13: **for** each pixel (i, j) (**parallel**) **do** ▷ Stage 2: anisotropic diffusion update
- 14: $D \leftarrow cN \cdot dN[k] + cS \cdot dS[k] + cW \cdot dW[k] + cE \cdot dE[k]$
- 15: $J[k] \leftarrow J[k] + 0.25 \cdot \lambda \cdot D$
- 16: **end for**
- 17: **end for**

Algorithm 5 Dense LU Decomposition & Solve Kernel

Require: $n \times n$ matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ (flat row-major), right-hand side $\mathbf{b} \in \mathbb{R}^n$
Ensure: Packed LU matrix $\mathbf{LU}$ (L strictly lower with unit diagonal, U upper triangle),
 pivot vector $\mathbf{piv} \in \mathbb{Z}^n$, solution $\mathbf{x} \in \mathbb{R}^n$

```

1:  $\mathbf{piv}[i] \leftarrow i$    for  $i = 0$  to  $n - 1$ 
2: for  $k = 0$  to  $n - 1$  do                                 $\triangleright$  LU Factorization with Partial Pivoting
3:   Find pivot row:  $p \leftarrow \arg \max_{i=k \dots n-1} |A_{ik}|$ 
4:   if  $p \neq k$  then
5:     Swap rows  $k$  and  $p$  in  $\mathbf{A}$ ; swap  $\mathbf{piv}[k]$  and  $\mathbf{piv}[p]$ 
6:   end if
7:    $akk \leftarrow A_{kk}$ 
8:   if  $|akk| < 10^{-15}$  then return singular
9:   end if
10:  for  $i = k + 1$  to  $n - 1$  do                                 $\triangleright$  Elimination below diagonal
11:     $A_{ik} \leftarrow A_{ik}/akk$                                  $\triangleright$  multiplier stored in L
12:     $\ell_{ik} \leftarrow A_{ik}$ 
13:    for  $j = k + 1$  to  $n - 1$  do
14:       $A_{ij} \leftarrow A_{ij} - \ell_{ik} \cdot A_{kj}$ 
15:    end for
16:  end for
17: end for
18:  $\mathbf{LU} \leftarrow \mathbf{A}$                                  $\triangleright$  now packed LU
19:                                 $\triangleright$  Forward substitution:  $\mathbf{Ly} = \mathbf{Pb}$ 
20: for  $i = 0$  to  $n - 1$  do
21:    $y_i \leftarrow b[\mathbf{piv}[i]]$ 
22: end for
23: for  $i = 1$  to  $n - 1$  do
24:   for  $j = 0$  to  $i - 1$  do
25:      $y_i \leftarrow y_i - \mathbf{LU}_{ij} \cdot y_j$ 
26:   end for
27: end for
28:                                 $\triangleright$  Back substitution:  $\mathbf{Ux} = \mathbf{y}$ 
29: for  $i = n - 1$  down to  $0$  do
30:    $x_i \leftarrow y_i$ 
31:   for  $j = i + 1$  to  $n - 1$  do
32:      $x_i \leftarrow x_i - \mathbf{LU}_{ij} \cdot x_j$ 
33:   end for
34:    $x_i \leftarrow x_i / \mathbf{LU}_{ii}$ 
35: end for

```

Algorithm 6 Sparse LU Decomposition & Solve Kernel

Require: CSR matrix $\mathbf{A}$ (row-major, sorted columns), right-hand side $\mathbf{b} \in \mathbb{R}^n$

Ensure: CSR matrices $\mathbf{L}$ (unit lower triangular), $\mathbf{U}$ (upper triangular), row permutation vector $\mathbf{P} \in \mathbb{Z}^n$, solution $\mathbf{x} \in \mathbb{R}^n$

```
1:  $\mathbf{P}[i] \leftarrow i$    for  $i = 0$  to  $n - 1$ 
2: Initialize work rows (dynamic sparse copies of  $\mathbf{A}$  rows)
3: for  $k = 0$  to  $n - 1$  do                                      $\triangleright$  LU Factorization with Partial Pivoting
4:   Find pivot row:  $p \leftarrow \arg \max_{i=k \dots n-1} |\text{work}_i[k]|$ 
5:   if  $p \neq k$  then
6:     Swap work rows  $k$  and  $p$ ; swap  $\mathbf{P}[k]$  and  $\mathbf{P}[p]$ 
7:   end if
8:    $\text{pivot} \leftarrow \text{work}_k[k]$ 
9:   if  $|\text{pivot}| < 10^{-14}$  then return singular
10:  end if
11:  Store  $\mathbf{U}[k, :]$  (all columns  $\geq k$  from work row  $k$ )
12:  for  $i = k + 1$  to  $n - 1$  do                                    $\triangleright$  Elimination below diagonal
13:     $a_{ik} \leftarrow \text{work}_i[k]$ 
14:    if  $|a_{ik}| < 10^{-15}$  then continue
15:    end if
16:     $\text{multiplier} \leftarrow a_{ik} / \text{pivot}$ 
17:    Append  $(k, \text{multiplier})$  to  $\mathbf{L}$  row  $i$ 
18:     $\triangleright \text{work}_i \leftarrow \text{work}_i - \text{multiplier} \cdot \text{work}_k$  (sparse)
19:    Clear sparse accumulator
20:    Scatter  $\text{work}_i$  into accumulator
21:    Scatter  $-\text{multiplier} \cdot \text{work}_k$  into accumulator
22:    Rebuild  $\text{work}_i$  from non-zeros in accumulator (sorted)
23:  end for
24: end for
25: Convert dynamic  $\mathbf{L}$  rows and  $\mathbf{U}$  rows to final CSR format  $\triangleright$  add unit diagonal to  $\mathbf{L}$ 
26:                                      $\triangleright$  Forward substitution:  $\mathbf{L}\mathbf{y} = \mathbf{P}\mathbf{b}$  (unit diagonal)
27: for  $i = 0$  to  $n - 1$  do
28:    $y_i \leftarrow b[\mathbf{P}[i]]$ 
29:   for each sub-diagonal entry  $(j, \text{val})$  in  $\mathbf{L}$  row  $i$  ( $j < i$ ) do
30:      $y_i \leftarrow y_i - \text{val} \cdot y_j$ 
31:   end for
32: end for
33:                                      $\triangleright$  Back substitution:  $\mathbf{U}\mathbf{x} = \mathbf{y}$ 
34: for  $i = n - 1$  down to  $0$  do
35:    $\text{sum} \leftarrow 0$ 
36:   for each entry  $(j, \text{val})$  in  $\mathbf{U}$  row  $i$  ( $j > i$ ) do
37:      $\text{sum} \leftarrow \text{sum} + \text{val} \cdot x_j$ 
38:   end for
39:    $x_i \leftarrow (y_i - \text{sum}) / U_{ii}$ 
40: end for
```

References

- [1] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. Rodinia: A benchmark suite for heterogeneous computing. In *IEEE International Symposium on Workload Characterization*, pages 44–54, 2009.
- [2] Quentin Ferro, Stef Graillat, Thibault Hilaire, and Fabienne Jézéquel. Performance of precision auto-tuned neural networks. In *2023 IEEE 16th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*, pages 592–599, 2023.
- [3] Stef Graillat, Fabienne Jézéquel, Romain Picot, François Févotte, and Bruno Lathuilière. Auto-tuning for floating-point precision with discrete stochastic arithmetic. *Journal of Computational Science*, 36:101017, 2019.
- [4] Fabienne Jézéquel, Sara sadat Hoseininasab, and Thibault Hilaire. Numerical validation of half precision simulations. In *Trends and Applications in Information Systems and Technologies*, pages 298–307, Cham, 2021. Springer International Publishing.
- [5] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. Seminal paper popularizing backpropagation for training multilayer neural networks.